

Un environnement à construire

Décisions, ordre, critères de réussite et retour arrière. Préparé ; aucune installation effectuée sur le Mac.

Recommandation : bâtir un socle Nix + Home Manager pour les CLI, garder Homebrew autonome pour les apps graphiques, essayer OmniWM seul, puis ajouter les compléments dont l'utilité est confirmée. SIP désactivé ouvre une branche yabai ; cela ne la rend pas prioritaire.

Profil et incertitudes

MacBook Pro 16, M5 Max, 64 Go, 2 To ; macOS 26.6 build 25G72 ; écran XDR intégré ; AZERTY Français standard, trackpad et défilement naturel. Ce sont les informations rapportées, pas des mesures faites ici. Lunar, Tailscale et TopNotch sont déjà présents. La résolution logique, les versions installées et les démarrages launchd restent à vérifier.

Point bloquant ciblé : 136 services auraient été désactivés par un preset « dev-minimal ». Son contenu et son origine ne sont pas établis. Aucun nouveau debloat ou rétablissement massif avant inventaire. Un dépôt candidat ne suffit pas à identifier le script exécuté. [S67]

Trois niveaux de décision

Niveau	Contenu	Règle
A - socle	Sauvegarde, Git, Nix/Home Manager, Homebrew, Ghostty, un WM	Recommandé ; une étape validée avant la suivante.
B - compléments	Raycast, Neru, BTT, contextes, monitoring, Finder	Essai ciblé ; conserver seulement une fonction utilisée.
C - spécialisé	nix-darwin, yabai/SA, SketchyBar, devshells, remappage avancé	Après stabilité ; besoin précis et retour arrière documenté.

Ce qui constitue la réussite

- Fenêtres et contenu défilent sans voler les gestes ; les raccourcis fonctionnent réellement sur AZERTY.
- Une source par paquet, un propriétaire par raccourci/configuration/service ; démarrage et veille fiables.
- Une palette visuelle lisible et cohérente ; aucun module résident sans utilité vérifiable.
- Configurations versionnées, sauvegarde restaurable et procédure de remplacement du WM.

Les numéros de version ci-dessous sont des instantanés de la recherche, pas des pins Homebrew. Après une mise à jour importante de macOS ou d'un WM, refaire les essais natifs.

Classement des installations

Ordre de déploiement recommandé ; les durées sont des estimations de travail, hors téléchargements et sauvegarde.

Ordre	Installer / mettre en place	Pourquoi et sortie attendue
0 • 45-90 min	Diagnostic, inventaire debloat, baseline	Comprendre l'état actuel avant d'attribuer une panne à un nouvel outil.
1 • variable	Time Machine + test de restauration ; Git	Retour arrière des données et trace des configurations. [S26]
2 • 45-90 min	Nix/Lix, Home Manager minimal ; Homebrew	CLI reproductibles, apps GUI simples à essayer. [S45-S46,S14]
3 • 30-60 min	Ghostty + shell ; CLI ; Atuin facultatif	Terminal agréable et tâches quotidiennes plus rapides. [S12-S13,S54]
4 • 1-2 h	OmniWM ; alternative Paneru	Tester la promesse centrale de tiling défilant. [S01-S03,S49-S50]
5 • 30-60 min	Raycast si valeur distincte	Extensions, snippets, quicklinks ; ne pas doubler WM/clipboard. [S51]
6 • 30-60 min	Neru ; puis essai BTT	Navigation sans souris et gestes utiles sans conflits. [S09-S11,S52]
7 • 45-90 min	Thème, bureaux nommés, contextes	Faire converger apparence et usages réels. [S16-S18,S32,S71]
8 • 30-60 min	Stats ; Finder ; modules ciblés	Observer et compléter les fonctions manquantes. [S53,S34]
9 • 1-3 h	nix-darwin ; branche yabai si besoin	Déclarer les réglages validés ; effets avancés après contrôle. [S29,S36-S41]

Budget et licences

Le socle Nix/Home Manager, Ghostty, OmniWM, Paneru, AeroSpace, yabai, Neru, Hammerspoon et Stats est libre et gratuit. OmniWM est GPL-2.0-only ; Home Manager et Paneru sont MIT. Raycast est propriétaire, avec une offre personnelle gratuite suffisante pour l'essai ; Pro est facultatif. BTT est propriétaire et payant, avec essai de 45 jours. Bunch est gratuit mais n'est pas présenté ici comme libre. [S03,S46,S49,S35,S40,S16,S53,S51-S52,S32]

Évaluer les outils payants avec une tâche concrète avant achat. Lunar est déjà installé : conserver la version actuelle tant qu'un besoin d'écran externe ou d'automatisation ne justifie pas autre chose. [S65-S66]

0-1 • État des lieux et sauvegarde

Première séance : lecture seule, puis protection des fichiers et des données.

Inventaire à conserver privé

```
bash bin/diagnostic.sh
command -v nix brew rg fzf zoxide bat jq starship
launchctl print-disabled gui/$(id -u)
launchctl print-disabled system
```

Le diagnostic fourni collecte des informations techniques sélectionnées. Les deux commandes launchctl listent des overrides de désactivation ; elles ne montrent pas tous les plist supprimés, réglages modifiés ou effets du debloat. Ne pas publier la sortie brute sans relecture.

- Retrouver le script réellement exécuté, sa révision, le preset et les journaux ou sauvegardes qu'il a produits. Comparer les overrides et fichiers modifiés avec ces preuves ; « dev-minimal » n'est pas une origine vérifiée. [S67]

- Relever les apps/version, éléments de connexion, launch agents/daemons tiers et installation Nix éventuelle. Ne pas confondre absence dans Login Items et absence de service.

- Tester les fonctions que les outils futurs utiliseront : Spotlight/recherche Finder, presse-papiers, Apple Shortcuts, Safari, notifications, réseau/Tailscale, veille/réveil, trackpad et microphone.

Si une fonction native échoue : établir le lien avec un changement identifié, sauvegarder sa valeur et rétablir seulement cet élément. Ne pas lancer une commande globale de « restore all ». Reporter une installation qui dépend de la fonction en panne.

Sauvegarde et baseline

- Configurer Time Machine sur une destination appropriée, terminer une sauvegarde et restaurer un fichier de test. Une génération Nix, Git ou Syncthing ne remplace pas ce test. [S26,S27]

```
bash bin/backup-configs.sh
bash bin/backup-configs.sh --apply
```

Le second appel copie une sélection de configurations dans un répertoire privé. Il ne sauvegarde ni tout le Mac, ni toutes les apps, ni le dépôt Nix. Des dotfiles shell peuvent contenir des secrets : ne pas verser ces copies dans Git.

- Garder un état de référence : mêmes apps et luminosité, courte période de repos puis de travail normal ; noter énergie/CPU dans Moniteur d'activité et qualité de la veille. On comparera ensuite dans les mêmes conditions.

Critère de passage

Les fonctions natives essentielles marchent ; une sauvegarde a été restaurée ; les changements de debloat restent identifiés ou consignés comme inconnus. Ne pas annoncer de gain de performance à partir du seul nombre de services désactivés.

2 • Nix dès le socle

Home Manager autonome d'abord ; nix-darwin pour le système plus tard.

Propriétaire	Périmètre initial	Limite
Nix + Home Manager	CLI, versions résolues par flake.lock ; shell ensuite	Pas de contrôle complet des apps GUI ni de macOS. [S46]
Homebrew autonome	Casks et quelques outils WM disponibles via Brew	Rolling release ; --no-upgrade n'est pas un lock. [S14]
App / fichier manuel	Réglages WM et gestes pendant les essais	Exporter après validation ; pas de symlink read-only imposé.
nix-darwin - tardif	Réglages/services système sélectionnés	Conflits avec debloat et propriétaire du daemon à examiner. [S29]

Installation et première construction

- S'il n'existe aucune installation Nix : suivre l'installateur Lix officiel et conserver son reçu et sa procédure de désinstallation. Lix est une implémentation compatible de Nix ; ne pas empiler les installateurs. Une installation existante requiert sa procédure de mise à niveau, pas une seconde installation. [S45,S29]
- Installer Homebrew depuis brew.sh si absent ; vérifier le préfixe natif /opt/homebrew. Garder cette installation indépendante de Nix pendant les essais.
- Placer le kit dans ~/Projects/macOS-config sans écraser un dépôt ; compléter nix/identity.nix. Initialiser Git et suivre les fichiers de configuration avant de construire une flake située dans le dépôt.

```
git init
git add .gitignore nix manifests configs bin *.txt *.tsv sources.json
cd nix
nix flake lock
nix build ".#homeConfigurations.mac.activationPackage" --out-link result-home
```

Le lock sera produit sur le Mac : aucun lock fictif n'est livré. La construction n'active pas le profil ; elle écrit le store, le lock et le lien résultat. Le modèle livré n'a pas été évalué par Nix dans cet environnement.

Activation et retour arrière

```
./result-home/activate
# Nouveau terminal :
command -v rg fzf zoxide bat jq starship home-manager
```

Comparer les origines des exécutables ; ne retirer un paquet Brew équivalent qu'après validation du remplacement. Ne pas forcer l'écrasement d'un fichier existant. Versionner flake.lock après relecture.

Retour : home-manager generations, choisir la génération précédente et exécuter son chemin d'activation. Ces générations restaurent les éléments gérés par Home Manager, pas les données personnelles. Ne pas nettoyer les anciennes générations pendant les essais. [S46]

3 • Terminal, shell et CLI

Un gain indépendant du choix du WM ; développer la configuration progressivement.

Ghostty et identité visuelle

```
bash bin/install-stage.sh terminal
bash bin/install-stage.sh --apply terminal
```

Ghostty 1.3.1 est le point de départ vérifié. Tester terminal autonome avant Quake intégré au WM. Le kit offre un thème opaque et un thème « glass » à 94 % d’opacité ; fusionner manuellement si des réglages existent déjà. Le flou concerne Ghostty, pas toutes les fenêtres macOS. [S12-S13,S69]

CLI Nix : un petit ensemble utile

Outil	Usage concret	Choix
ripgrep + jq	Chercher rapidement dans du texte ; inspecter du JSON	Socle, sans remplacer aveuglément grep/sed système.
fzf + zoxide	Retrouver un fichier/une entrée ; rejoindre un dossier fréquent	Socle, intégration shell après sauvegarde. [S58-S59]
bat + Starship	Lire du code ; prompt indiquant dossier/Git/contexte	Socle ; prompt léger et lisible. [S57]
Atuin	Historique interrogeable par contexte et réutilisation	Option, local d’abord ; pas de compte ni sync automatique. [S54]
Yazi + Lazygit	Parcourir des fichiers ; relire commits/diffs dans le terminal	Seulement si réellement utilisés. [S55-S56]

Reprendre le shell, sans écraser son environnement

- Sauvegarder .zshrc/.zshenv/.zprofile et éventuel zsh XDG ; relever les lignes d’intégration Lix/Nix et Homebrew. Dans nix/profile.nix, activer enableShell puis reconstruire. Les fichiers déjà présents peuvent bloquer l’activation : les comparer et migrer explicitement.


```
cd nix
home-manager switch --flake .#mac
```
- Atuin possède Ctrl-R si activé : le modèle désactive alors l’intégration Zsh de fzf pour éviter le conflit. fzf reste disponible en ligne de commande. Les flèches normales restent conservées. [S54,S58]
- L’historique de commandes peut contenir des tokens ou chemins sensibles : définir exclusions/rétention avant toute synchronisation. Ne pas versionner sa base.

Critère de passage

Nouveau terminal, terminal lancé depuis Finder et après redémarrage : PATH cohérent ; commandes trouvées ; historique sans conflit ; prompt rapide. Retour par génération Home Manager et copies des fichiers précédents, selon ce qui a été repris.

4 • Un seul gestionnaire de fenêtres

Deux défilements distincts : colonnes/fenêtres et contenu dans les apps.

Candidat	Adéquation	Décision
OmniWM 0.7.6	Tiling défilant, Overview, barre, palette et Quake ; macOS 26+ Apple Silicon	Premier essai ; jeune projet, validation native nécessaire. [S01-S03,S68]
Paneru 0.5.2	Colonnes défilantes, Spaces natifs ; configuration TOML ou Lua	Deuxième essai si OmniWM pose problème ou si Spaces natifs prioritaires. [S49-S50]
AeroSpace 0.21.0-Beta	Arbre de tiling, bureaux émulés, orientation clavier	Alternative sobre ; pas de défilement de colonnes. [S35]
yabai 7.1.25	BSP et Spaces natifs ; effets via scripting addition	Branche spécialisée, détaillée plus loin. [S36-S41]

OmniWM : configuration initiale minimale

```
bash bin/install-stage.sh --apply core
```

- Ouvrir manuellement, accorder Accessibilité et Surveillance de l'entrée requises. Enregistrement de l'écran seulement pour Overview/prévisualisations qui le demandent. Accès complet au disque n'est pas un prérequis global : Hidden Bar requiert macOS 27, hors profil 26.6. [S01]
- Conserver « les écrans disposent de Spaces distincts ». Désactiver la réorganisation automatique par utilisation. Pour OmniWM, commencer avec un Space natif sur cet écran ; les bureaux de travail sont ceux du WM. Ne pas supprimer des Spaces ou fenêtres sans les déplacer. [S01]
- Créer Focus / Web / Messages / Atelier. Deux doigts restent au contenu ; colonnes à trois ou quatre doigts. Réserver l'autre geste aux bureaux si possible, puis vérifier les conflits Mission Control. En 0.7.6, fingerCount=2 rejette tout le fichier. [S03]
- Configurer les raccourcis depuis l'interface : Hyper = Ctrl+Alt+Cmd ; flèches focus, Shift déplacement ; W/E/D/T bureaux ; O Overview. Pas de remappage Caps Lock automatique. Exporter une configuration valide après essais.

App et CLI doivent avancer ensemble : 0.7.6 utilise IPC 19. Pas de mélange avec un ancien omniwmctl. Tester la commande intégrée avant d'ajouter des scripts. [S03,S08]

Essai Paneru et règle de changement

Quitter OmniWM et retirer son démarrage avant Paneru. Installer core-paneru ; suivre le README officiel et démarrer en premier plan avant d'ajouter le service. Revoir les raccourcis par défaut, notamment Cmd-H. Choisir TOML ou Lua. Aucun preset Paneru inventé n'est livré. [S49-S50]

Pour chaque candidat : Safari, Discord, Ghostty, boîtes de dialogue, plein écran, fermeture, veille/réveil, Spaces et gestes. Garder le premier qui passe les tâches réelles ; ne pas changer pour un benchmark non comparable.

5-6 • Lanceur, clavier et gestes

Ajouter une fonction après avoir identifié son propriétaire.

Raycast : à tester après le WM

L'intérêt est dans les extensions, snippets, quicklinks et actions rapides. L'offre personnelle gratuite suffit pour commencer ; son historique de copie est limité à trois mois dans cette offre. Ne pas prendre Pro pour construire le socle. [S51]

```
bash bin/install-stage.sh --apply launcher
```

- Essayer d'abord une ouverture d'app, un quicklink de recherche, un snippet récurrent et une extension relue. Si retenu : Cmd-Space pour Raycast, déplacer Spotlight vers Alt-Space avant attribution ; libérer les combinaisons concurrentes.

- Désactiver les raccourcis de window management Raycast si OmniWM les possède. Choisir **un seul** historique de copie : OmniWM, Raycast ou Maccy. Exclure les apps sensibles et définir rétention/effacement. [S05,S22,S51]

Neru : naviguer dans les interfaces

```
bash bin/install-stage.sh --apply navigation
```

Neru 1.57.0 : hints, grille et défilement sans souris. Hyper-N/G/S dans le kit ; cinq raccourcis par défaut explicitement désactivés. Accessibilité requise ; les modes vision/contour peuvent demander Screen Recording. Fusionner le fragment TOML, ne pas écraser une configuration entière. [S09-S11]

- Tester Safari, Finder et Discord : certaines interfaces ont un arbre d'accessibilité incomplet ; la grille reste un secours. Si Secure Input retient les raccourcis, identifier l'app qui le détient plutôt que désactiver une protection globale. [S03,S44]

BetterTouchTool : une phase d'essai, pas un doublon

```
bash bin/install-stage.sh --apply gestures
```

Essai de 45 jours ; garder BTT uniquement si des gestes personnalisés ou chaînes d'actions réduisent des manipulations répétées. Définir après les gestes OmniWM/macOS. Ne pas lui confier en parallèle le WM, le clipboard et Hyper. [S52]

- Premier exemple : geste ou bouton explicite pour lancer un contexte, deuxième pour une action fréquente d'app. Tester d'abord au clavier ; un geste ne doit pas intercepter le défilement deux doigts ni les swipes du WM.

Remappage avancé : facultatif

Karabiner-Elements devient pertinent pour des couches et remaps conditionnels. Le réserver à un besoin que les raccourcis suffisent mal à résoudre ; valider lettres, accents, login et clavier externe avant d'étendre. Un seul remappeur possède Caps Lock/Hyper. [S70]

7 • Apparence et contextes

Rendre l'environnement cohérent, sans multiplier les moteurs résidents.

Une palette, une barre, des réglages mesurés

- Commencer avec le thème « Bleu Nuit » livré : fond sombre, accent bleu, bordure active discrète, espacement choisi dans le WM. Réduire le bruit : barre compacte et icônes utiles seulement. Lisibilité prioritaire sur transparence.
- Utiliser bordures/barre OmniWM d'abord. JankyBorders et SketchyBar servent une branche sans ces fonctions ou un besoin de barre scriptable : ne pas les cumuler avec leurs équivalents intégrés. SwiftBar convient à quelques scripts de statut ponctuels. [S24,S42-S43]
- Dock : choisir position/taille/masquage dans les réglages système ; garder le délai natif pendant l'essai. Finder : barre de chemin/état, extensions et dossiers favoris. TopNotch reste un choix esthétique existant, pas un composant nécessaire au tiling.
- Exporter les configurations et captures du résultat validé. Les dotfiles publics inspirent une palette ou une idée ; ne pas importer leur installateur, leurs services et leurs chemins personnels en bloc.

Contextes : choix selon la complexité

Moteur	Quand le choisir	Premier scénario
Shortcuts natif	Actions système et apps avec actions natives	Focus « Travail », ouvrir les apps/URL utiles. [S71]
Bunch	Liste lisible d'apps/fichiers et ouverture/fermeture	Contexte Travail / Communication ; gratuit, non qualifié de libre. [S32]
Hammerspoon	Logique Lua et pilotage précis du WM	Hyper-P : Focus du workspace + ouverture des ressources. [S16-S18]

Choisir un moteur de contexte principal. BTT/Raycast peuvent le déclencher ; ils n'ont pas besoin de réimplémenter le même scénario.

Projet Hammerspoon préparé dans le kit

Module projects facultatif ; les projets sont désactivés tant que noms/chemins ne sont pas adaptés. Copier les modules retenus dans ~/.hammerspoon après sauvegarde, puis ajouter un require explicite dans init.lua. Le backend par défaut est OmniWM ; yabai est une alternative. Le module attend le succès du focus avant d'ouvrir les ressources, mais ne rétablit pas toute une disposition de fenêtres.

Avec OmniWM, vérifier le chemin du CLI et IPC 19 ; avec yabai, les Spaces doivent porter les labels attendus. Ne pas l'activer avec Paneru/AeroSpace : ces backends ne sont pas implémentés dans le kit. [S08,S17-S18]

8 • Les autres axes à fort potentiel

Modules conditionnels : chacun doit résoudre un manque observé.

Domaine	Action concrète	Déclencheur / prudence
Finder et recherche	Tags, dossiers intelligents, Quick Look ; Syntax Highlight pour le code [S34]	Avant extension, vérifier l'indexation suite au debloat ; tester previews.
Navigateur et web	Profils Safari séparés, groupes d'onglets ; quicklinks Raycast	Isoler travail/personnel sans ajouter automatiquement un second navigateur.
Capture, OCR, voix	Capture native et Live Text ; dictée native ; TypeWhisper local si insuffisant [S20-S21]	Tester français/bruit/latence, microphone et modèle local ; contrôler option premium.
Audio / visio	Presets entrée/sortie, Shortcuts si actions disponibles	Matériel et problème précis avant utilitaire audio supplémentaire.
Écrans	Conserver Lunar ; BetterDisplay si HiDPI/virtuel/écran externe [S65-S66,S72]	Écran interne seul : aucun besoin établi de réinstaller BetterDisplay.
Transfert / sync	LocalSend interplateformes ; Syncthing seulement pour dossiers définis [S30,S27]	LocalSend si Windows/Android ; sync propage aussi suppressions et erreurs.
Sauvegarde avancée	restic vers une seconde destination, clé protégée [S60]	Après Time Machine testé ; restaurer depuis la deuxième destination.
Réseau et accès distant	Tailscale existant ; LuLu si règles sortantes nécessaires [S31]	Vérifier clients/services existants et règles avant d'ajouter un filtre.
Monitoring / énergie	Stats avec quelques modules, intervalle modéré ; bottom en TUI [S53,S64]	Comparer au repos sans module ; pas de modification des ventilateurs.
Développement	Devshell Nix + nix-direnv ; mise comme alternative [S47,S61]	Un propriétaire par runtime ; relire .envrc avant allow.
Maintenance / apps	Inventaire versions, exports, désinstallation via propriétaire	Éviter nettoyeur automatique, boost RAM ou « optimisation » globale.

Installer seulement le module GUI correspondant quand le besoin apparaît : quicklook, dictation, transfer, monitoring ou network. Les anciens modules maccy/quicksilver/swiftbar/bar sont des alternatives ; les modules cli/backup/sync Brew sont désormais vides, ces paquets relevant du profil Nix.

Capture permanente de l'écran ou indexation de tout l'historique personnel : hors socle. Un besoin de recherche rétrospective doit justifier les permissions, rétention, destination et consommation avant tout essai. [S28]

9 • Déclaratif système et variantes

Approfondir après les usages stables ; Nix est intégré dès le début, pas repoussé à cette phase.

nix-darwin : élargir le périmètre, un réglage à la fois

- Après inventaire debloat : relever la valeur actuelle d'un réglage, le déclarer, construire, inspecter les changements puis activer. Commencer par defaults identifiés ; ne pas recréer la liste des 136 services dans launchd. [S29]
- Vérifier qui gère le daemon Nix et son implémentation. Le défaut nix-darwin utilise Nix upstream ; si Lix est retenu, déclarer le choix explicitement avant de lui déléguer cette responsabilité. Home Manager peut rester autonome. [S29,S45]
- Homebrew reste indépendant dans le parcours recommandé. nix-homebrew gère Homebrew/taps, pas à lui seul la déclaration des casks. Les installations Homebrew gérées par Nix, notamment nix-darwin ou nix-homebrew, relèvent du Tier 3 officiel ; ce coût de support doit être accepté avant migration. [S48,S15]
- Une config WM gérée par un lien vers le Nix store est en lecture seule. Tant que l'interface graphique écrit les réglages, garder un fichier mutable et exporter les versions. Éviter de promettre une configuration GUI « déclarative » incompatible avec sa sauvegarde.

Développement et dotfiles : choisir une méthode

Devshell Nix par projet pour environnements reproductibles ; nix-direnv pour l'activation par dossier, après revue de .envrc. mise est plus simple pour certains workflows de versions : choisir une autorité pour Node/Python/etc., pas deux qui se disputent le PATH. [S47,S61]

chezmoi est une alternative crédible pour des dotfiles à templates et plusieurs machines, si l'on préfère conserver une configuration mutable hors Home Manager. Ne pas lui faire gérer les mêmes fichiers. Ici, Git + Home Manager est le choix par défaut ; chezmoi n'est donc pas installé en plus. [S63]

Variantes cohérentes

Profil	Composition
Léger	Git + Homebrew + Ghostty ; Nix CLI petit ; un WM ; Spotlight ; gestes natifs.
Recommandé	Profil léger + shell HM ; Raycast si utile ; Neru ; contextes natifs ou Hammerspoon ; palette commune ; Stats limité.
Poussé	Profil recommandé + devshells ; nix-darwin ciblé ; restic seconde destination ; gestes BTT retenus ; barre scriptable seulement si besoin.
Spaces / effets	Remplacer le WM par yabai ; éventuellement SA puis JankyBorders/SketchyBar ; ne pas conserver les résidents concurrents.

Niri sert d'inspiration pour le tiling défilant ; c'est un compositor Wayland, pas un WM macOS à installer. Paneru et OmniWM représentent les pistes natives à comparer pour cet usage. [S62,S49,S03]

Branche yabai • SIP déjà désactivé

Option spécialisée ; désactivation SIP, SSV scellé et scripting addition sont des états distincts.

Pourquoi ne pas en faire le premier choix

yabai 7.1.25 est la release stable vérifiée. Le changelog master mentionne un correctif add_space pour macOS 26.6 qui n'est pas dans cette release ; l'issue associée confirme le problème traité. Ne pas promettre une création automatisée de Spaces stable sur le profil actuel ni compiler master sans décision explicite. [S38,S41]

Chemin de mise en place

- Arrêter OmniWM/Paneru/AeroSpace et supprimer leur lancement automatique ; garder leurs configs. Installer core-yabai. Commencer sans SA et tester tiling, focus et règles sur les apps réelles.
- Créer les Spaces manuellement et attribuer Focus / Web / Messages / Atelier aux quatre Spaces existants ; désactiver la réorganisation automatique. Le preset yabairc est à adapter au nombre et à l'ordre réels, pas à exécuter aveuglément.
- Si Hammerspoon retenu : sélectionner backend yabai et charger le module de touches. Hyper-W/E/D/T, Shift pour transfert ; flèches focus/réinsertion, F flottant, Return zoom. Vérifier AZERTY et labels sur le Mac.
- Charger la scripting addition seulement pour les fonctions qui l'exigent, selon le wiki de la version installée : état SIP exact, conditions Apple Silicon/ABI, chargement et éventuelle autorisation sudo limitée. Le kit ne modifie ni root, boot-args, sudoers ni SSV. [S36,S39-S40]
- Animations : Enregistrement de l'écran ; opacité et certaines fonctions exigent la SA. Une mise à jour du binaire peut demander une nouvelle injection et une actualisation de la règle sudoers fondée sur son hash. [S39-S40]

Essai réversible des effets

```
bash bin/yabai-effects.sh
bash bin/yabai-effects.sh --apply
# Retour avec le chemin de sauvegarde imprimé :
bash bin/yabai-effects.sh --restore "/chemin/yabai-effects.XXXXXX"
```

Le script capture cinq valeurs avant application, puis tente leur restauration en cas d'échec. La restauration de valeurs ne restaure pas l'organisation des fenêtres ; conserver le TSV et vérifier les effets. Cette branche de mutation est relue, pas exécutée sur macOS ici.

Abandon de la branche

Restaurer les valeurs/configs voulues, arrêter le service yabai, retirer son démarrage et suivre la désactivation de sa SA documentée ; relancer un seul autre WM. Le SIP désactivé ne justifie pas d'ajouter une exception système pour chaque outil. Les autres composants du socle fonctionnent sans sa désactivation.

Validation, démarrage et entretien

Un feu vert se fonde sur les usages du Mac, pas seulement sur la syntaxe des fichiers.

À tester	Critère de passage	En cas d'échec
CLI / shell	Nouveau terminal + login : origines PATH attendues ; historique fonctionnel	Génération HM précédente et comparaison des fichiers shell.
Fenêtres / contenu	Safari/Discord/Ghostty : colonnes et contenu indépendants ; dialogues utilisables	Quitter WM ; restaurer config ; tester alternative seule.
AZERTY / raccourcis	Chaque touche physique, accents et raccourcis d'app ; pas de doublon	Réattribuer une lettre/combinaison ; ne pas importer des chiffres US.
Veille / Spaces	Trois cycles veille/réveil, plein écran, fermeture/réouverture	Désactiver démarrage ; isoler le module ajouté.
Énergie / fluidité	Sessions comparables, mêmes apps/luminosité ; pas de hausse durable inexplicquée	Comparer avec complément quitté ; réduire preview/fréquence/effets.
Sauvegarde / contexte	Fichier restauré ; projet ouvre uniquement les bonnes ressources	Corriger chemins/labels ; contexte reste désactivé.
Démarrage	Un seul WM, launcher, clipboard, remappeur et moteur de contextes voulu	Retirer doublons Login Items/launchd/services ; relancer.

Ordre de stabilisation

Après chaque phase, faire au moins une session normale. N'activer le lancement automatique que pour un composant testé ; conserver l'inventaire de ses méthodes de démarrage. Laisser quelques jours d'usage au WM avant d'investir dans une barre complexe ou de grandes règles par app.

Mettre à jour séparément Nix inputs et apps Brew. Revoir les release notes, sauvegarder les réglages, conserver le lock précédent ; construire avant activation. OmniWM et son CLI se mettent à jour ensemble. Homebrew ne verrouille pas les versions GUI. Après update macOS : refaire les tests WM/permissions/veille ; yabai/SA exige une attention particulière. [S03,S14,S39]

Ce que le kit vérifie réellement

Syntaxe Bash/POSIX, TOML Neru, compilation Lua et simulations d'orchestration, modules Brew simples, aperçus sans mutation, refus sous Linux, cohérence archive/dossier. Ce sont des vérifications du livrable, pas une certification de macOS 26.6, de TCC, du rendu ou de l'IPC natif.

À valider sur le Mac : Nix evaluation/build/activation, résolution Homebrew, permissions, AZERTY, gestes, fenêtres, énergie, démarrage et sauvegarde. Le fichier verification.txt donne les contrôles exécutés et leurs limites. Aucun installateur global « tout d'un coup » n'est fourni.

Sources officielles

Liens cliquables. Consultation : 7-8 octobre 2026 ; les dates exactes par lien figurent dans sources.json.

[S01] OmniWM - compatibilité

<https://omniwm.app/help/known-limitations/>

[S03] OmniWM 0.7.6 - release, IPC 19 et gestes

<https://github.com/OmniNull/OmniWM/releases/tag/v0.7.6>

[S05] OmniWM - palette et presse-papiers

<https://omniwm.app/features/command-palette/>

[S08] OmniWM - actions CLI

<https://omniwm.app/reference/cli/actions/>

[S09] Neru - configuration

<https://github.com/y3owk1n/neru/blob/main/docs/reference/configuration.md>

[S11] Neru - versions

<https://github.com/y3owk1n/neru/releases>

[S12] Ghostty - ordre des configurations

<https://ghostty.org/docs/config>

[S13] Ghostty - options visuelles

<https://ghostty.org/docs/config/reference>

[S14] Homebrew - Bundle

<https://docs.brew.sh/Brew-Bundle-and-Brewfile>

[S15] Homebrew - niveaux de support

<https://docs.brew.sh/Support-Tiers>

[S16] Hammerspoon - projet et licence

<https://github.com/Hammerspoon/hammerspoon>

[S17] Hammerspoon - hs.task

<https://www.hammerspoon.org/docs/hs.task.html>

[S18] Hammerspoon - hs.urlevent

<https://www.hammerspoon.org/docs/hs.urlevent.html>

[S20] TypeWhisper - coeur libre et premium

<https://www.typewhisper.com/en/>

[S21] TypeWhisper - installation

<https://github.com/TypeWhisper/typewhisper-mac>

[S22] Maccy - projet et licence

<https://github.com/p0deje/Maccy>

[S24] SwiftBar - projet et licence

<https://github.com/swiftbar/SwiftBar>

Sources officielles • suite

Liens cliquables. Consultation : 7-8 octobre 2026 ; les dates exactes par lien figurent dans sources.json.

[S26] Apple - sauvegarde Time Machine

<https://support.apple.com/guide/mac-help/back-up-files-mh35860/mac>

[S27] Syncthing - attributs et sauvegardes

<https://docs.syncthing.net/users/faq.html>

[S28] Screenpipe - licence actuelle

<https://github.com/screenpipe/screenpipe/blob/main/LICENSE.md>

[S29] nix-darwin - instructions officielles

<https://github.com/nix-darwin/nix-darwin>

[S30] LocalSend - projet

<https://github.com/localsend/localsend>

[S31] LuLu - installation

<https://objective-see.org/products/lulu.html>

[S32] Bunch - gratuite

<https://bunchapp.co/>

[S34] Syntax Highlight - aperçus

<https://github.com/sbarex/SourceCodeSyntaxHighlight>

[S35] AeroSpace - alternative et limites

<https://github.com/nikitabobko/AeroSpace>

[S36] yabai - fonctions et SIP

<https://github.com/asmvik/yabai/wiki/Disabling-System-Integrity-Protection>

[S38] yabai - changelog et correctif non publié

<https://github.com/asmvik/yabai/blob/master/CHANGELOG.md>

[S39] yabai - installation et scripting addition

[https://github.com/asmvik/yabai/wiki/Installing-yabai-\(latest-release\)](https://github.com/asmvik/yabai/wiki/Installing-yabai-(latest-release))

[S40] yabai - référence des commandes

<https://github.com/asmvik/yabai/blob/master/doc/yabai.asciidoc>

[S41] yabai - création de Space sous 26.6, issue 2799

<https://github.com/asmvik/yabai/issues/2799>

[S42] JankyBorders - code, licence et installation

<https://github.com/FelixKratz/JankyBorders>

[S43] SketchyBar - installation

<https://felixkratz.github.io/SketchyBar/setup>

[S44] skhd - Secure Keyboard Entry et permissions

<https://github.com/asmvik/skhd>

Sources officielles • suite

Liens cliquables. Consultation : 7-8 octobre 2026 ; les dates exactes par lien figurent dans sources.json.

[S45] Lix - installation

<https://lix.systems/install/>

[S46] Home Manager - socle utilisateur

<https://github.com/nix-community/home-manager>

[S47] nix-direnv - environnements de projet

<https://github.com/nix-community/nix-direnv>

[S48] nix-homebrew - rôle et limites

<https://github.com/zhaofengli/nix-homebrew>

[S49] Paneru - projet et installation

<https://github.com/karinushka/paneru>

[S50] Paneru - formule 0.5.2

<https://formulae.brew.sh/formula/paneru>

[S51] Raycast - offre et limites

<https://www.raycast.com/pricing>

[S52] BetterTouchTool - gestes et essai

<https://folivora.ai/>

[S53] Stats - monitoring

<https://github.com/exelban/stats>

[S54] Atuin - historique local et synchronisation optionnelle

<https://github.com/atuinsh/atuin>

[S55] Yazi - gestionnaire de fichiers

<https://github.com/sxyazi/yazi>

[S56] Lazygit - interface Git

<https://github.com/jesseduffield/lazygit>

[S57] Starship - prompt

<https://starship.rs/>

[S58] fzf - recherche interactive

<https://github.com/junegunn/fzf>

[S59] zoxide - navigation shell

<https://github.com/ajeetdsouza/zoxide>

[S60] restic - sauvegardes

<https://github.com/restic/restic>

[S61] mise - runtimes de développement

<https://mise.jdx.dev/>

Sources officielles • suite

Liens cliquables. Consultation : 7-8 octobre 2026 ; les dates exactes par lien figurent dans sources.json.

[S62] Niri - compositor Wayland, pas WM macOS

<https://github.com/niri-wm/niri>

[S63] chezmoi - alternative pour dotfiles

<https://github.com/twpayne/chezmoi>

[S64] bottom - monitoring TUI

<https://github.com/ClementTsang/bottom>

[S65] Lunar - projet

<https://github.com/alin23/Lunar>

[S66] Lunar - site et tarifs

<https://lunar.fyi/>

[S67] Debloat - dépôt candidat, origine non établie

<https://github.com/OleksandrKrupko/mac-os-debloat>

[S68] OmniWM - cask Apple Silicon/macOS 26

<https://formulae.brew.sh/cask/omniwm>

[S69] Ghostty - release 1.3.1

<https://ghostty.org/docs/install/release-notes/1-3-1>

[S70] Karabiner-Elements - remappage

<https://karabiner-elements.pqrs.org/>

[S71] Apple - Shortcuts

<https://support.apple.com/guide/shortcuts-mac/intro-to-shortcuts-apdf22b0444c/mac>

[S72] BetterDisplay - projet et options

<https://github.com/waydabber/BetterDisplay>